

COMP1009

Programming Paradigms

Lecture OO14:
Design Patterns
Module Summary

Patterns

- There are usually many good ways to solve a problem
 - Some are usually better than others
 - Most ‘small’ problems that we try to solve have been solved before
- Patterns help us to identify ways in which a problem can be solved
 - We identify the pattern for the problem
 - Then look at how people have solved this in the past
 - Much less ‘thinking’ and ‘working out’
 - Avoid potential pitfalls later on
 - Good solutions are designed to enable modification later₂

Responsibilities and good design

- If a class has multiple types of responsibilities, it is usually better to split it into multiple classes
- The strategy pattern allows you to 'switch out' combinations of implementations of responsibilities by changing which object is responsible for doing them (e.g. layout of contents)
- The observer pattern allows other classes to react to what is happening – i.e. move the handling of this into another class
 - E.g., another class may react when a button is pressed
- The iterator pattern allows you to 'iterate' through something
 - e.g., contents of an object
- All three of these are behavioural patterns – defining/changing behaviour of an object
- There are other types of patterns too, e.g. creational
- Note: MVC (model-view-controller) is actually more of an architectural pattern, but we will treat it as a 'design pattern'

Some Final Patterns (1)

Singleton

Creational patterns

- Creational patterns are patterns related to creation of objects
 - i.e. not just the usual `'new MyClass()'`
- We will *briefly* consider two simple ones:
 - Singleton – only one instance of this class
 - Factory – wrap up the creation of another object(s), particularly when you don't know what to create
- Key element of both:
 - Use a method to create the object, not new directly

Singleton Overview

- Singleton: single object
 - You create one instance
 - When it is first needed
- Whenever anything wants an instance (object), give it the same one
- This instance itself can do whatever you want
- Basic Singleton in Java uses a static method
 - A static member is associated with the class itself
 - i.e. not associated with any object, no 'this' object reference
 - And one object, with a static object reference to it

Singleton class

oneObject : Singleton

getObject() : Singleton

Static members usually underlined

Example Singleton

```
public class Singleton
{
    private static Singleton oneObject = null;

    private Singleton() {} // HIDDEN! Only this class!

    static Singleton get() // Get the one object
    {
        if ( oneObject == null )
            oneObject = new Singleton();
        return oneObject;
    }

    public void foo() {} // Some method...
}
```

Example code to use Singleton

```
public class MainTest
{
    public static void main(String[] args)
    {
        Singleton s1 = Singleton.get();
        s1.foo();

        Singleton s2 = Singleton.get();
        s2.foo();

        if ( s1 == s2 )
            System.out.println("Objects are identical!");

        Singleton s = new Singleton(); // Error!
    }
}
```


Alternative (simpler) Singleton

- This is a worse implementation, because it creates the object even if you never ask for one. If asked for an implementation use the other one

```
public class SimplerSingleton
{
    private static SimplerSingleton oneObject =
        new SimplerSingleton();

    private SimplerSingleton() {} // HIDDEN!

    static SimplerSingleton get() // Get the one object
    { // Object now already exists
        return oneObject;
    }

    public void foo() {} // Some method...
}
```

Aside: two for different purposes

```
package singleton;

public class TwoObjectsForDifferentThings
{
    private static TwoObjectsForDifferentThings one = null;
    private static TwoObjectsForDifferentThings two = null;

    // Private constructor prevents manual construction
    private TwoObjectsForDifferentThings() {}

    static TwoObjectsForDifferentThings get1()
    { // E.g. get object to manage config file
        if ( one == null )
            one = new TwoObjectsForDifferentThings();
        return one;
    }

    static TwoObjectsForDifferentThings get2()
    { // E.g. get object to manage log file
        if ( two == null )
            two = new TwoObjectsForDifferentThings();
        return two;
    }

    public void foo() {} // Some method...
}
```

Singleton summary

- Static members are associated with the class rather than an object – i.e. shared between all objects
- We can take advantage of this to have only a single object of a class – if we already created an **object** then **re-use** that one
- Wrap the object creation in a **static method**
 - Optionally synchronized to avoid issues of concurrency/multi-threading
- Keep a track of the one object created in a static member variable
- **Important:** Never create an object of the class manually
 - You can enforce this by making a private constructor
 - To create an object you need a constructor which matches the parameters supplied
- Another request for an object of this class object will return the same object
 - You can even keep requesting it and getting the same one

Singleton vs Static methods

- Sometimes it is important to have an object
 - E.g. where we need to pass an object into a function
 - Where we need an object which implements an interface
- You could have made foo() static and not used the singleton pattern here. Compare:

```
public void foo() {}  
Singleton.get().foo();
```

```
public static void foo() {}  
Singleton.foo();
```

- But then you lose the advantage that you can pass around an object
- Having an 'object' can be really important sometimes (trust me?)
"All problems in computer science can be solved by another level of indirection" – objects allow us to add a level of indirection in OO

Some Final Patterns (2)

Factory

Factory

- A Factory is an object which manufactures other objects (i.e., a factory makes things)
- We will concentrate on simple examples, although there are more complex and powerful examples of this
- Importantly, because a factory creates the object for you, you don't need to know which precise sub-class that factory object will create – you can let the factory decide
- The main concept of a 'factory' is the manufacturing of objects

Simple factory – for animals

```
public class AnimalFactory
{
    public Animal createLion()
    {
        return new Lion();
    }

    public Animal createFish()
    {
        return new Fish();
    }
}
```

Using the factory

```
public class TestFactory1
{
    public static void main( String[] args )
    {
        // Create factory
        AnimalFactory f = new AnimalFactory();

        // Use factory to create object
        Animal a = f.createFish();

        System.out.println( a.getName() );
    }
}
```


Why might factories be useful

- Controlling the creation of different objects can have a number of uses, e.g:
 - If you want to hide the actual classes that you use, so users always deal with them as if they are base class objects
 - In the previous example we did not care that there was a Fish object – we only ever treated it as an Animal object
 - It gives you the extra level of indirection on object creation that may be useful (to solve problems)
 - Or if you want to do something else when objects are created – for example ...

Counting the number of objects created

```
public class AnimalFactory  
{
```

```
    public int count = 0;
```

```
    public Animal createLion() { ++count; return new Lion(); }
```

```
    public Animal createFish() { ++count; return new Fish(); }
```

```
    public int getAnimalCount()
```

```
{
```

```
    return count;
```

```
}
```

```
}
```

Notice that the return type is `Animal` regardless of which object is created

Because construction always comes through here, we can keep a count, or even keep a list of animals created

More complex factories

- You can also use more complex versions of the factory pattern
- Here we knew which type of factory object we had, and what it created
- We could instead have a base class factory object reference without knowing what it creates...

An abstract factory

```
public interface BaseAnimalFactory  
{  
    Animal create();  
}
```

```
public class FishFactory implements BaseAnimalFactory  
{  
    @Override  
    public Animal create()  
    {  
        return new Fish();  
    }  
} // and similar for Lion, Bird, etc
```

Using the abstract factory

```
public class TestFactory3
{
    public static void useFactory( BaseAnimalFactory f )
    {
        // Create one – we don't know the animal type
        Animal a = f.create();
        System.out.println( a.getName() );
        // Create 5 more of the same type
        for ( int i = 0; i < 5 ; i++ )
            System.out.println( f.create().getName() );
    }

    public static void main( String[] args )
    {
        BaseAnimalFactory f = new LionFactory();
        useFactory( f );
        // And another one, without keeping a reference to factory
        useFactory( new FishFactory() );
    }
}
```

Comparison

Singleton vs Factory

- Both the factory and the singleton had a method for creating objects for you
 - A static method on the class to create for a singleton
 - Another object to create them for a factory
- Simple factory:
 - Created object was of a different class to the factory
 - Could be of the same class, or a clone
- Abstract factory:
 - Could create an interface (or class) for ‘factory’
 - And subclasses implement create in different ways
 - I used this for infinite/finite list creation for FP in Java
 - Create a list that does ..., hiding the implementation classes
- Singleton has a static member function/method
 - Always returns the same object
 - Usually also supports the useful functionality

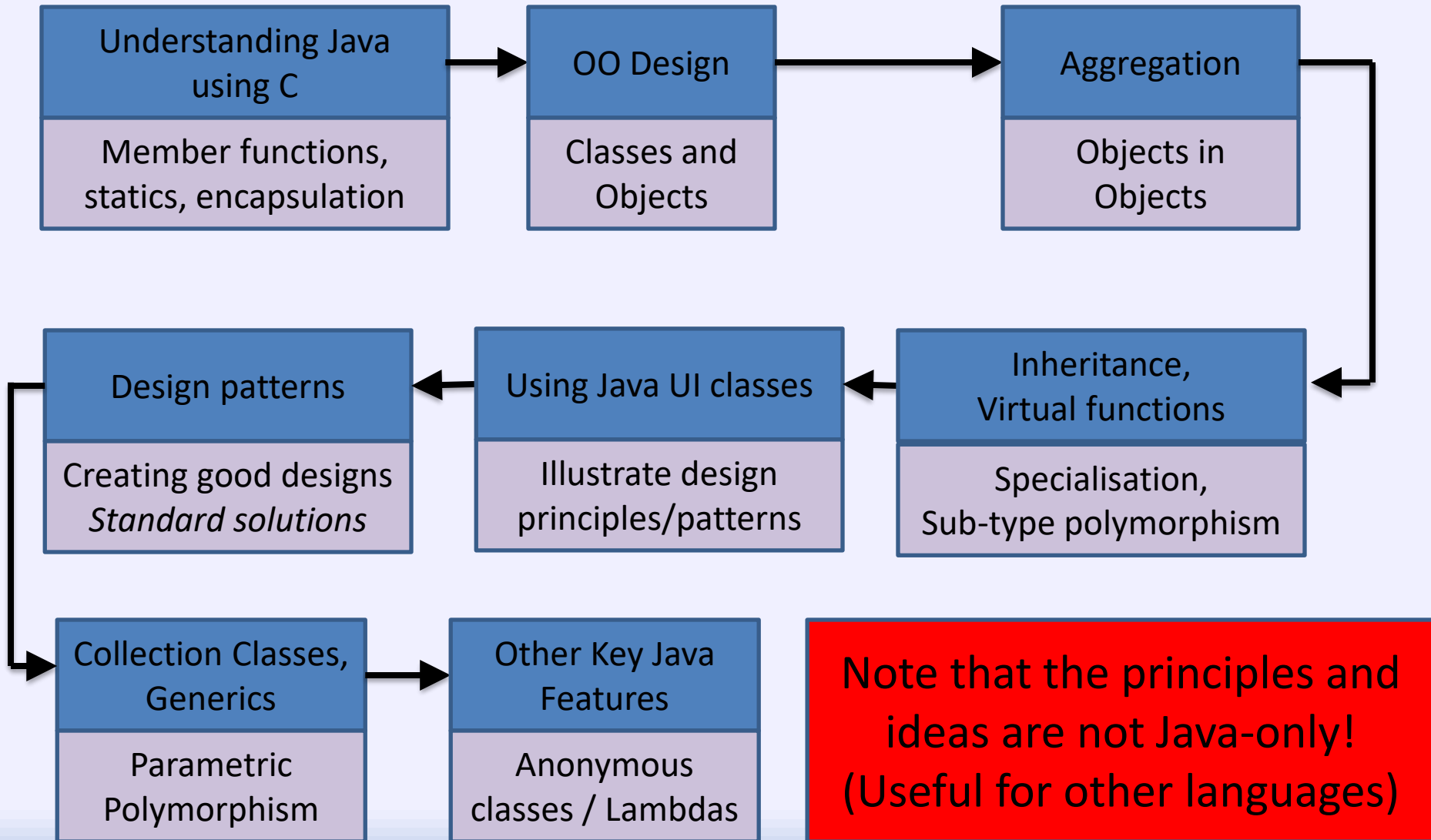
Design Patterns Summary

OO Design Patterns

- There are **many** different design patterns for OO
- We saw a few common ones on this course:
 - **Observer** : tell me when something happens
 - **Strategy** : lets another object modify the behaviour of an object (e.g. layout manager, or infinite list/container)
 - **Iterator** : lets you iterate through the contents
 - **Model-View-Controller (MVC)**: separate the responsibilities of data storage, presentation to user, and control logic
 - **Singleton** : create just one instance and use it anywhere
 - **Simple Factory** : delegate the job of choosing which class to actually create to another object (which could, for example, produce a sub-class object)
- There are many more patterns
 - Serious OO programmers get to know them
 - These are good ways to solve known tasks

Module 00 Summary

Initial Outline



From Lecture 1: What is an OO language

- A language which promotes or allows object oriented programming and design
- Usual features:
 - Objects (and classes)
 - Encapsulation (methods and data together)
 - **Data hiding (restrict access to data)**
 - Composition/aggregation (has-a)
 - Inheritance (specialisation, reuse, 'is-a')
 - Abstraction (and interfaces)
 - Polymorphism (dynamic dispatch, late binding)
 - Behaviour only known at runtime

We used all of these features

- Objects (and classes)
 - Objects are the things we create using new
 - Classes specify what is in an object and what we can do to them
- Encapsulation (methods and data together)
 - You put the methods which act on the data in the class
- Abstraction (and interfaces)
 - We only need to worry about what methods so, not how they are implemented
- **Data hiding (restrict access to data)**
 - Ideally we make data private and the 'interface' methods public - **next slide**
- Composition/aggregation (has-a)
 - Objects can contain other objects – and use them to do the work
- Inheritance (specialisation, reuse, 'is-a')
 - We used this a lot – reusing things like JButton and JLabel behaviour
- (Sub-type) Polymorphism (dynamic dispatch, late binding)
 - Behaviour only known at runtime – we used this a lot too
 - It's also the reason that Lambdas can 'wrap up' a function
- We also used generics (parametric polymorphism)
 - Which in Java are **implemented using** sub-type polymorphism

Data Hiding

- Create an object
- Store the reference (pointer) in a variable

```
MyClass ob = new MyClass();
```

- Call a method on the object

```
ob.myMethod();
```

- Usually we should not access data directly

```
ob.val = 4; // Don't do this
```

```
ob.setVal(4); // Do this...
```

```
int val = ob.getVal(); // or this
```

Access permissions – repeated

- Packages are very important because of access privileges
- Attributes and methods are labelled with access permissions
- **public**
 - Anything can access this
- **protected**
 - This class + this package + subclasses can access this
- ***nothing (package)***
 - This class + this package can access this
- **private**
 - Only this class can access this
- **So, there is a hierarchy of importance:**
 - The class itself has the most access (any modifier)
 - Then other classes in this package (blank, public, protected)
 - Then subclasses (public or protected)
 - Then 'world' (has to be public)

The problem of Java packages

- There are two problems in Java with demonstrating access permissions in simple programs:
 1. classes in the same package have increased access privileges – so you need to use different packages to see the effects
 2. main() is a class member – so has special access
- Usually you want to think of:
 - Private data/methods: hidden inside this class
 - Protected data/methods: hidden but also available to sub-classes
 - Public: available for everyone to use
- Java classes in the same package can access all except private members – which breaks the idea of data hiding
- To see how data hiding and encapsulation should work we need to put classes in different packages

Problem of main() being a class member

```
public class MainAccessIssue
{
    private int p; // Private data!

    public static void main(String[] args)
    {
        MainAccessIssue ob = new MainAccessIssue();
        ob.p = 4;
        System.out.println( "" + ob.p );
    }
}
```

- Because main() is a class member (even though static) the compiler is quite happy to let us alter the private data
- This hides the relevance of private data when we write small programs **and put main in the same class**

Object Oriented Language Summary

- Objects (and classes)
 - Objects are the things we create using new
 - Classes specify what is in an object and what we can do to them
- Encapsulation (methods and data together)
 - You put the methods which act on the data in the class
- Abstraction (and interfaces)
 - We only need to worry about what methods so, not how they are implemented
- Data hiding (restrict access to data)
 - Ideally we make data private and the 'interface' methods public
- Composition/aggregation (has-a)
 - Objects can contain other objects – and use them to do the work
- Inheritance (specialisation, reuse, 'is-a')
 - We used this a lot – reusing things like JButton and JLabel behaviour
- (Sub-type) Polymorphism (dynamic dispatch, late binding)
 - Behaviour only known at runtime – we used this a lot too
 - It's also the reason that Lambdas can 'wrap up' a function
- We also used generics (parametric polymorphism)
 - Which in Java are **implemented using** sub-type polymorphism

Remaining Lectures/Resources

- Tuesday, 10am, Auditorium, in-person Final lecture
 - Wrap-up, joint lecture (Graham on-line)
- Friday, 11am, Auditorium, in-person, Revision Lecture
 - Recorded. (Last day of term – unless a vote says otherwise)
- Optional exercises (replace lectures of the past)
 - Warning: could confuse you if you are not confident, but useful as ‘extension exercises’ if you are happy with OO
- Optional exercise 1 – FP in Java
 - How are infinite lists managed? Can Java do them?
 - Why this style is used in Haskell, but not in Java
 - What Haskell is having to do ‘behind the scenes’ (quite a lot)
- Optional exercise 2 – make your program do multiple things at once – threading (complex but useful)